



Chapitre 5:

Les types composés

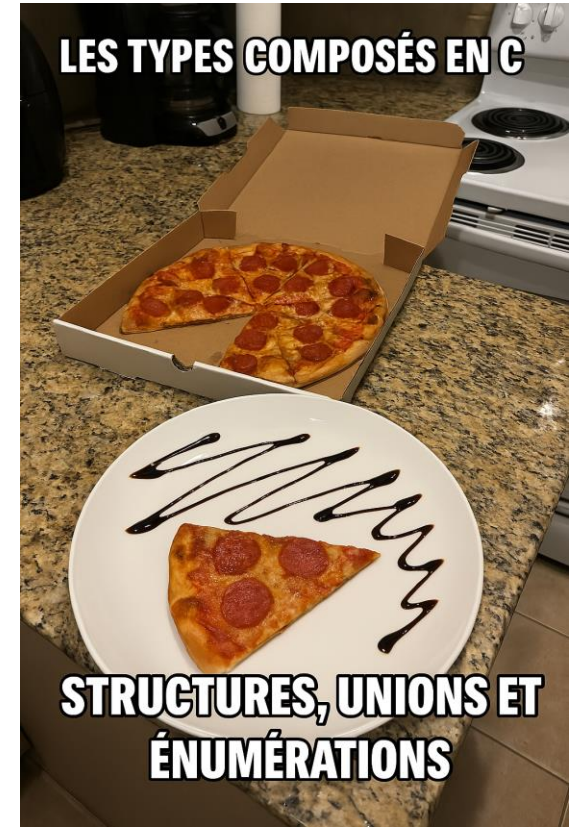
Babacar DIOP

Adresse Mail: diopbabacar4888@gmail.com

Téléphone: 777812040

Introduction

- ❖ Le langage C fournit plusieurs types de base : int, float, char, etc.
- ❖ Cependant, pour représenter des données complexes, il est nécessaire de regrouper plusieurs valeurs de types éventuellement différents.
- ❖ Pour cela, le C permet de définir de nouveaux types appelés types composés, parmi lesquels on trouve:
 - Les structures (struct)
 - Les unions (union)
 - Les énumérations (enum)



I. Les structures

I.1 Définition & Déclaration

- ❖ Une **structure** permet de regrouper plusieurs variables, éventuellement de types différents, sous un même nom.
- ❖ Chaque variable dans la structure est appelée **champ** ou **membre**.

- ❖ **Syntaxe:**

Methode 1: Déclaration simple avec nom de structure

```
struct Personne {  
    char nom[20];  
    char prenom[20];  
    int age;  
};  
  
struct Personne p1, p2; // Déclaration de deux variables
```

Methode 2: Déclaration avec nom de structure et variables intégrées

```
struct Personne {  
    char nom[20];  
    char prenom[20];  
    int age;  
} p1, p2;
```

Methode 3: Déclaration avec typedef

```
typedef struct {  
    char nom[20];  
    char prenom[20];  
    int age;  
} Personne;  
  
Personne p1, p2; // Plus besoin de struct
```

I. Les structures

I.2. Accès aux membres

On utilise l'opérateur **point .** pour accéder aux membres :

```
p1.age = 30;

printf("Âge : %d\n", p1.age);

p2.nom[0] = 'X';
```

I.3. Initialisation d'une structure

Exemple 1: Initialisation directe à la déclaration

```
Personne p = {"Ndiaye", "Moustapha", 45};
```

Exemple 2: Initialisation après déclaration

```
Personne p;
strcpy(p.nom, "Moustapha");
strcpy(p.prenom, "Lamine");
p.age = 35;
```

I.4. Affectation entre structures

```
Personne p1 = {"Ali", "Fall", 28};
Personne p2;

p2 = p1; // Tous les champs de p1 sont copiés dans p2
```

I.5. Limites : Comparaison impossible directement

```
if (p1 == p2) // ✗ Erreur
```

Il faut comparer champ par champ.

II. Les Énumérations

Définition & Déclaration

- ❖ Une **énumération** permet de définir un type limité à un ensemble de **valeurs symboliques**.

```
enum Jour {LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE};
```

- ❖ *Par défaut, les valeurs sont associées à des entiers consécutifs à partir de 0*

```
LUNDI = 0, MARDI = 1, ..., DIMANCHE = 6
```

- ❖ **Syntaxe:**

```
enum Jour j1, j2;  
  
j1 = LUNDI;  
j2 = MARDI;  
  
printf("%d\n", j1); // Affiche 0
```

Il est aussi possible de déclarer des variables directement :

```
enum Jour {LUNDI, MARDI, MERCREDI} d1, d2;
```